StreamCab

Real-Time Taxi Fare Intelligence
Kafka · Spark Structured Streaming · XGBoost

Bibek Dhungana, Xiaodi Shao, Raivath Mallela

April 22, 2026

Big Data Application

Agenda

1. **Overview** — problem statement and system architecture
2. **Data** — source, collection, schema, and metadata
3. **Process** — end-to-end pipeline inputs and outputs
4. **Results** — dashboard, model quality, and insights
5. **Takeaways** — what worked and what was hard

Overview

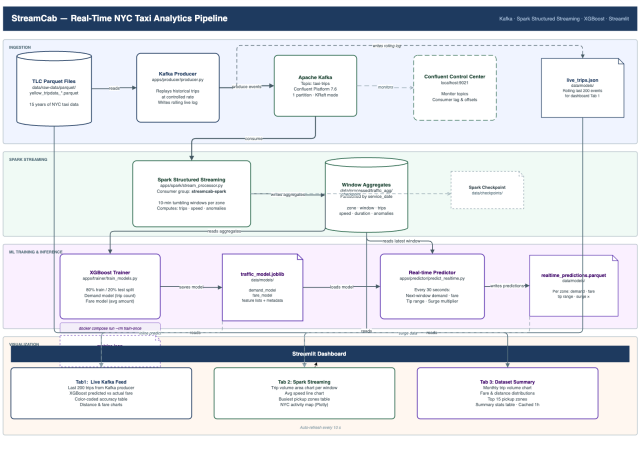
Problem Statement

NYC taxi demand and pricing shift rapidly by zone and time.

- Static batch reports lag behind operational decisions.
- No live view of which zones are surging right now.
- Tip prediction from historical patterns alone is too slow.

Goal: Build a full streaming analytics and prediction stack — ingestion to live dashboard — that runs reproducibly on a single machine.

System Architecture



Parquet → Kafka → Spark → PostgreSQL → Predictor → Dashboard

Architecture Components

- **Producer + Kafka:** replay trips as a live event stream
- **Consumer:** write raw events to `live_trips`
- **Spark:** 10-min windowed zone aggregates
- **PostgreSQL:** live trips, aggregates, predictions
- **Trainer:** XGBoost on historical aggregates
- **Predictor:** periodic inference every 30 s
- **Dashboard:** live KPIs, map, zone rankings

Data

Data Source and Collection

Source: NYC TLC Yellow Taxi Trip Records — public monthly Parquet files
`nyc.gov/site/tlc/about/tlc-trip-record-data.page`

Collection workflow

1. Data Size: **152GB** CSV and **24GB** parquet
2. Tentative Rows: **1642819405** Around 16 billion
3. Download monthly parquet files via project script (retry + back-off).
4. Store under `/data/raw-data/parquet/`.
5. Producer replays rows to Kafka as pseudo-live events.

Scale: 2010–2025 supported; default pull 2022-01 → 2024-12; quick-start uses 2022-01 → 2022-03 for fast local iteration.

Reference data: TLC zone centroid CSV for spatial zone mapping.

Schema and Metadata

Raw event fields

- `pickup_datetime`, `dropoff_datetime`
- `passenger_count`, `trip_distance`
- `pu_location_id`, `do_location_id`
- `total_amount`

Windowed aggregate fields

- `window_start`, `window_end`
- `pu_location_id`, `trip_count`
- `avg_trip_distance`, `avg_duration_min`
- `avg_speed_mph`, `avg_total_amount`

Coordinates snapped to nearest zone centroid via BallTree ($\approx$100 m precision).

Data Quality Rules

Rows are retained only when they satisfy all bounds:

Field	Accepted range
Duration	1–180 min
Trip distance	0.2–100 mi
Average speed	1–80 mph
Fare / total amount	$0–$500
Pickup zone	valid ID or mappable lat/lon

Conservative filtering reduces outlier influence and improves model stability.

Process

End-to-End Processing Pipeline

1. **Ingest** — read parquet, normalize timestamps and zone IDs.
2. **Stream** — replay rows to Kafka at configurable rate.
3. **Aggregate** — Spark applies quality filters; computes 10-min window stats per zone.
4. **Persist** — consumer writes window stats to a table.
5. **Train** — XGBoost reads aggregates in small batches.
6. **Predict** — predictor scores real-time windows.
7. **Visualize** — Streamlit renders KPIs, zone rankings, and map.

Inputs and Outputs

Inputs

- Historical Parquet trip files
- Zone centroid CSV

Intermediates

- Kafka topic `taxi-trips`
- `traffic_agg` table
- Spark checkpoints

Final outputs

- `traffic_model.joblib`
- `metrics.json`
- `predictions` table
- Live dashboard

Model Training Configuration

Target: `tip_amount`

Features:

- `location_id`, `hour`
- `day_of_week`
- `trip_distance`, `duration_min`
- `speed_mph`, `trip_count`

XGBoost settings:

- `max_depth=5`, `lr=0.05`
- 80/20 temporal train-test split

Strategy: 500K-row parquet mini batches;

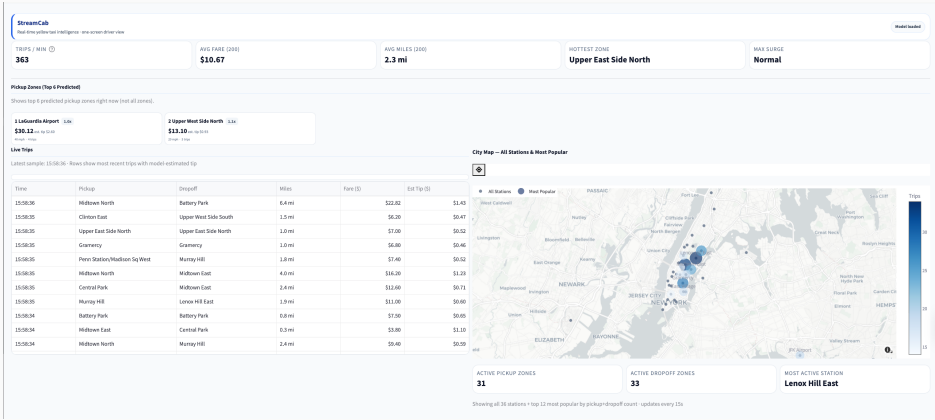
Results

Model Performance

Model	MAE ($)	MAPE (%)
Baseline (zone-hour mean)	1.99	15.43
XGBoost (test)	**0.49**	**3.42**

75% reduction in MAE over the baseline. XGBoost captures nonlinear interactions between demand, speed, duration, and time that a simple mean cannot represent.

Dashboard and Analytics Outputs



Live KPIs · zone rankings with surge badges · NYC map · rolling trip table

Key Takeaways

What Worked and What Was Hard

Worked well

- Reproducible one-command startup with Docker Compose.
- Spark windowed aggregates as stable, interpretable features.
- Strong prediction accuracy with XGBoost even with 10% training data.

Most difficult

- **XGboost** performance drop when we train in all data. May be over-fitting. (Pytorch might be easy)
- Network communication across docker container and volume mounts.
- Schema changes and inconsistent data. LC format changes (lat/lon vs. zone IDs).
- Difficult to restart training on failures. Spark checkpoints helps but difficult.

Additional References

- **Youtube links:** https://youtu.be/lEEw0fPl7UM
- **Code Links:** https://github.com/dhunganabibek/StreamCab
- **Data source:** www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Questions?

Thank you.